Intelligent spell checker using Levenshtein Edit Distance

ON

Dictionary; misspelled word

Project report submitted in partial fulfilment of the requirements for the Course

DATA STRUCTURES AND ALGORITHMS - 3

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026 27

by

Name : Rohini Tanmayi **roll 2520030349**

Name : S Manvi Vaishnavi **roll 2520030326**

Name : Pramod Krishna Macha **roll 2520030311**

Under the Esteemed guidance of

Anuradha CH

Professor, CSE

COMPUTER SCIENCE ENGINEERING



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF CSE

Name of Title: intelligent spell checker using Levenshtein Edit Distance

- **ABSTRACT :**

The Intelligent Spell Checker is a software application designed to identify misspelled words and suggest the most appropriate corrections using the Levenshtein Edit Distance algorithm. The system compares an input word with words stored in a dictionary and calculates the minimum number of insertions, deletions, and substitutions required to transform one word into another. Based on the calculated distance, the system ranks possible corrections according to their similarity and presents the most relevant suggestions to the user.

The application is designed to support multiple languages and Unicode text, making it suitable for diverse users and multilingual applications. To handle large dictionaries efficiently and minimize response time, optimized searching and ranking techniques can be incorporated. This project demonstrates the practical application of dynamic programming, particularly in solving string-matching and text processing problems, while providing an efficient and user-friendly solution for automatic spelling correction.

• INTRODUCTION :

In the modern digital world, text-based communication is an essential part of education, business, social media, and everyday activities. While typing, users often make spelling mistakes due to typing errors, unfamiliar words, or incorrect word usage. An intelligent spell checker can help users identify these errors and provide suitable corrections quickly and accurately.

The *Intelligent Spell Checker* is a software system developed to detect misspelled words and suggest the most similar correct words from a dictionary. The system uses the Levenshtein Edit Distance algorithm, which calculates the minimum number of insertions, deletions, and substitutions required to transform one word into another. The words with the smallest edit distance are considered the most suitable suggestions.

The proposed system is designed to support multiple languages and Unicode text, allowing it to process a wide range of characters and languages. Since dictionaries can contain thousands or millions of words, the system also focuses on improving search efficiency and reducing response time.
This project demonstrates the practical use of Dynamic Programming in solving real-world problems. By combining string comparison, dictionary searching, similarity ranking, and optimization techniques, the Intelligent Spell Checker provides an efficient solution for automatic spelling correction and improves the overall accuracy of digital text input.

PROBLEM STATEMENT :

Manual proofreading is slow and error-prone, and most real-world typos are not random — they follow predictable patterns such as adjacent-key mistakes, missing or extra letters, and letter swaps. The problem addressed by this project is: given a possibly misspelled word and a dictionary of valid words, efficiently identify whether the word is misspelled and, if so, return a ranked list of the most likely intended words — accurately, quickly even against large dictionaries, and across multiple languages and Unicode scripts.

Formally, for an input word w and dictionary D, the system computes the edit distance d(w, x) for each candidate word $x \in D$, and returns the subset with the smallest distances, ranked by similarity and tie-broken by heuristics such as word frequency.

THEORETICAL GROUNDING :

- Origin: The algorithm is named after Vladimir Levenshtein, who introduced it in 1965 for correcting binary codes; it was later generalized to arbitrary strings.
- Category: It belongs to the family of edit distance metrics, which are themselves a subset of string similarity and alignment algorithms — the same dynamic programming idea underlies sequence alignment in bioinformatics, such as the Needleman-Wunsch algorithm.
- Why Dynamic Programming applies: The problem exhibits optimal substructure, since the best way to transform a prefix of one string into a prefix of another builds on optimal solutions to smaller prefixes, and overlapping subproblems, since the same prefix pairs are reused across the matrix — the two conditions that justify dynamic programming over brute-force recursion.
- Metric properties: Levenshtein distance is a true metric — it satisfies non-negativity, symmetry, identity of indiscernibles, and the triangle inequality — which is why it is trusted for ranking and similarity tasks rather than being an ad hoc heuristic.
- Relation to the course: This project is a direct application of the Dynamic Programming unit of Data Structures and Algorithms - 3, connecting general DP principles such as optimal substructure and memoization versus tabulation to a practical text-processing problem

• DESIGN OF THE PROJECT :

The Intelligent Spell Checker is designed as a text-processing system that accepts an input word, compares it with words in a dictionary, calculates Levenshtein Edit Distance, and ranks the closest matches. The design focuses on accurate suggestions, efficient dictionary searching, and support for multiple languages and Unicode text.

Component	Function	Output
Input	Accepts the word entered by the user.	Input word
Dictionary	Provides valid words for comparison.	Candidate words
Levenshtein Distance	Calculates insertions, deletions and substitutions.	Edit distance
Ranking	Orders candidates from most to least similar.	Ranked suggestions
Output	Displays the closest corrections.	Suggested words

FLOWCHART :



• REFERENCES :

1. Levenshtein, V. I. (1966). Binary Codes Capable of Correcting Deletions, Insertions and Reversals. Soviet Physics Doklady, 10(8), 707–710.
2. Wagner, R. A., & Fischer, M. J. (1974). The String-to-String Correction Problem. Journal of the ACM, 21(1), 168–173.
3. Damerau, F. J. (1964). A Technique for Computer Detection and Correction of Spelling Errors. Communications of the ACM, 7(3), 171–176.

DESIGN, METHODOLOGY AND TECHNICAL SOUNDNESS

The project is designed as a text-processing system that accepts a possibly misspelled word, compares it with candidate words in a dictionary, calculates Levenshtein Edit Distance, and ranks the closest matches.

Project methodology

- Input: Accept the word entered by the user.
- Dictionary: Provide valid candidate words for comparison.
- Edit distance: Calculate the required insertions, deletions and substitutions.
- Ranking: Order candidates from most to least similar.
- Output: Display the closest correction suggestions.

The methodology follows the project flow already documented in the original report and directly applies Dynamic Programming to the string-comparison problem.

Why Dynamic Programming fits

- Theoretical grounding identifies optimal substructure: a best transformation between prefixes can be built from smaller prefix solutions.
- It also identifies overlapping subproblems: the same prefix comparisons can recur, making reuse of computed results valuable.
- This makes Dynamic Programming more suitable than repeatedly solving the same subproblems with brute-force recursion.

OBJECTIVE

- To identify misspelled words by comparing the user's input with words stored in a dictionary.
- To calculate the Levenshtein Edit Distance between the input word and dictionary words using Dynamic Programming.
- To suggest the most appropriate corrections by ranking dictionary words according to their similarity to the input.
- To handle common spelling errors involving insertions, deletions, and substitutions.
- To improve spelling correction efficiency by using optimized searching and ranking techniques for large dictionaries.
- To support multiple languages and Unicode text, making the system suitable for multilingual applications.
- To demonstrate the practical application of Dynamic Programming in a real-world string-matching and text-processing problem.